

Computer Programming 143 – Lecture 15

Arrays III

Faculty of Engineering



Copyright

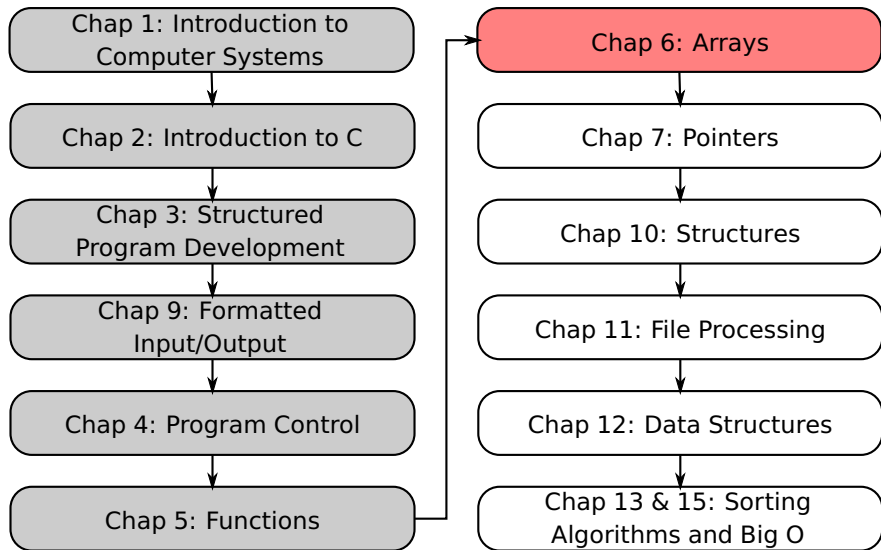
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

1 Sorting Arrays (6.8)

2 Case Study: Computing Mean, Median and Mode (6.9)

6.8 Sorting Arrays

Sorting data

- Important computing application
- Virtually every organization must sort some data

Bubble sort

- Several passes through the array
- Successive pairs of elements are compared
 - If increasing order (or identical), no change
 - If decreasing order, elements are exchanged
- Repeat

NOTE: This probably is the daftest algorithm on the planet!

6.8 Bubble Sort

c[0]	13	pass = 1 i = 0
c[1]	4	
c[2]	1	
c[3]	12	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 0
c[1]	13	
c[2]	1	
c[3]	12	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 1
c[1]	13	
c[2]	1	
c[3]	12	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 1
c[1]	1	
c[2]	13	
c[3]	12	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 2
c[1]	1	
c[2]	13	
c[3]	12	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 2
c[1]	1	
c[2]	12	
c[3]	13	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 3
c[1]	1	
c[2]	12	
c[3]	13	
c[4]	3	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 3
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	13	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 4
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	13	
c[5]	7	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 4
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	13	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 5
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	13	
c[6]	11	

6.8 Bubble Sort

c[0]	4	pass = 1 i = 5
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	4	pass = 2 i = 0
c[1]	1	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 0
c[1]	4	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 1
c[1]	4	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 2
c[1]	4	
c[2]	12	
c[3]	3	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 2
c[1]	4	
c[2]	3	
c[3]	12	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 3
c[1]	4	
c[2]	3	
c[3]	12	
c[4]	7	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 3
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	12	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 4
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	12	
c[5]	11	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 4
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 2 i = 5
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 0
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 1
c[1]	4	
c[2]	3	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 1
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 2
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 3
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 4
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort

c[0]	1	pass = 3 i = 5
c[1]	3	
c[2]	4	
c[3]	7	
c[4]	11	
c[5]	12	
c[6]	13	

6.8 Bubble Sort: Example

Problem

- Write a C function to perform bubble sorting of an array

Pseudocode

```
begin bubbleSort(array, arraySize)  
  for pass from 1 to arraySize - 1  
    for index from 0 to (arraySize - 2)  
      if array[index] > array[index + 1]  
        swap values in array[index] and array[index + 1]  
end
```

6.8 Bubble Sort: Example

```
#include <stdio.h>
#define SIZE 10
void bubbleSort( int b[], int size );// function prototype
int main() {
    int a[SIZE] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };//initialize a
    int j; //array index

    printf( "Data items in original order\n" );
    for (j = 0; j < SIZE; j++) {
        printf("%d ",a[j]);
    }
    bubbleSort( a, SIZE );//sorting the array

    printf( "\nData items after sort\n" );
    for (j = 0; j < SIZE; j++) {
        printf("%d ",a[j]);
    }
    return 0;
}
```

```

/* bubble sort */
void bubbleSort( int b[], int size ) {
    int i;      /* inner counter */
    int pass;   /* outer counter */
    int hold;   /* temporary location used to swap elements */

    /* loop to control number of passes */
    for ( pass = 1; pass < size; pass++ ) {
        /* loop to control number of comparisons per pass */
        for ( i = 0; i < size - 1; i++ ) {
            /* compare adjacent elements and swap them if first
               element is greater than second element */
            if ( b[i] > b[i + 1] ) {
                hold = b[i];
                b[i] = b[i + 1];
                b[i + 1] = hold;
            }
        }
    }
}

```

6.9 Case Study: Computing Mean, Median and Mode

Computing mean, median and mode

- Given an unsorted array of integers
- The **mean** is the average value of the array
 - Determined by dividing the sum of the elements by the number of elements
- The **median** is the “middle value” of the array
 - Determined by sorting the array and picking the element at the middle index
- The **mode** is the value that occurs most frequently
 - Determined by counting the number of times each value occurs
 - The result is also stored in an array
 - The mode is given by the maximum value of this array of results

6.9 Case Study: Computing Mean, Median and Mode

Problem

- Ninety nine responses were collected in a survey. Each response is a number between 1 and 9. Place the responses in an array and summarize the results by giving the mean, median and mode of the survey data.

6.9 Case Study: Computing Mean, Median and Mode

Pseudocode

```
begin mean(array)  
  initialize total to 0  
  for index from 0 to arraySize - 1  
    add array[index] to total  
  calculate mean as total/arraySize  
end  
  
begin median (array)  
  display unsorted array  
  use function bubbleSort to sort array  
  display sorted array  
  display median element  
end
```


6.9 Case Study: Computing Mean, Median and Mode

Pseudocode

```
begin mode(array)  
    initialize variables largest, modeValue and freqArray elements to 0  
    count the number of occurrences of each number → freqArray  
    for each occurrence rating from 1 to 9  
        if freqArray[rating] > largest  
            assign freqArray[rating] to largest  
            set modeValue to rating  
    Display the modeValue and its number of occurrences  
end  
begin main()  
    initialize response array  
    call function mean[response array]  
    call function median[response array]  
    call function mode[response array]  
end
```

6.9 Case Study: Computing Mean, Median and Mode

```
// fig06_13.c adapted from Dietel and Dietel
#include <stdio.h>
#define SIZE 99
// function prototypes
void mean(const int answer[], int size);
void median(int answer[], int size);
void mode(int freq[], const int answer[], int size) ;
void bubbleSort(int b[], int size);
void printArray(const int a[], int size);
// function main begins program execution
int main(void) {
    int frequency[10] = {0}; // initialize array frequency
```

```

// initialize array response
int response[SIZE] =
{6, 7, 8, 9, 8, 7, 8, 9, 8, 9, 7, 8, 9, 5, 9, 8, 7, 8, 7, 8,
6, 7, 8, 9, 3, 9, 8, 7, 8, 7, 7, 8, 9, 8, 9, 8, 9, 7, 8, 9,
6, 7, 8, 7, 8, 7, 9, 8, 9, 2, 7, 8, 9, 8, 9, 8, 9, 7, 5, 3,
5, 6, 7, 2, 5, 3, 9, 4, 6, 4, 7, 8, 9, 6, 8, 7, 8, 9, 7, 8,
7, 4, 4, 2, 5, 3, 8, 7, 5, 6, 4, 5, 6, 1, 6, 5, 7, 8, 7};
// process responses
mean(response, SIZE);
median(response, SIZE);
mode(frequency, response, SIZE);
return 0;
}

```

```

void mean(const int answer[], int size) {
    printf("%s\n%s\n%s\n", "-----", " Mean", "-----");
    int total = 0; // variable to hold sum of array elements

    // total response values
    for (size_t j = 0; j < size; ++j) {
        total += answer[j];
    }

    printf("The mean is the average value of the data\n"
        "items. The mean is equal to the total of\n"
        "all the data items divided by the number\n"
        "of data items (%u). The mean value for\n"
        "this run is: %u / %u = %.4f\n\n",
        size, total, size, (double) total / size);

    return;
}

```

```

void median(int answer[], int size) {
    printf("\n%s\n%s\n%s\n%s", "-----", " Median", "-----",
        "The unsorted array of responses is");
    printArray(answer, size); // output unsorted array

    bubbleSort(answer, size); // sort array

    printf("%s", "\n\nThe sorted array is");
    printArray(answer, size); // output sorted array
    // display median element
    printf("\n\nThe median is element %u of\n"
        "the sorted %u element array.\n"
        "For this run the median is %u\n\n",
        size / 2, size, answer[size / 2]);

    return;
}

```

```

void mode(int freq[], const int answer[], int size) {
    printf("\n%s\n%s\n%s\n", "-----", " Mode", "-----");

    // summarize frequencies
    for (size_t j = 0; j < size; ++j) {
        ++freq[answer[j]];
    }
    int largest = 0;    // represents largest frequency
    int modeValue = 0; // represents most frequent response

    for (size_t rating = 1; rating <= 9; ++rating) {

        // keep track of mode value and largest frequency value
        if (freq[rating] > largest) {
            largest = freq[rating];
            modeValue = rating;
        }
    }
}

```

```

    // display the mode value
    printf("\nThe mode is the most frequent value.\n"
        "For this run the mode is %d which occurred %d times.\n",
        modeValue, largest);

    return;
}

```

```

// output array contents (20 values per row)
void printArray(const int a[], int size) {
    // output array contents
    for (size_t j = 0; j < size; ++j) {
        if (j % 20 == 0) { // begin new line every 20 values
            printf("\n");
        }
        printf("%2d", a[j]);
    }
    return;
}

```

Today

Arrays III

- Sorting arrays
- Case study: mean, median and mode

Next lecture

Arrays IV

- Multidimensional Arrays

Homework

- 1 Study Sections 6.8-6.9 in Deitel & Deitel
- 2 Do Self Review Exercise 6.1(d) in Deitel & Deitel
- 3 Do Exercises 6.14 in Deitel & Deitel